

# Campus event reservation project

## Supporting document for Nora Patel

SYNTHETIC DEMO MATERIAL | All people, projects and results are fictional.

### Problem

A fictional campus club needs an API that reserves seats without overselling when multiple students book at the same time. Repeated requests must not create duplicate reservations.

### Design

Store events, reservations and idempotency keys in PostgreSQL. Lock the event row inside a short transaction, verify remaining capacity, then create the reservation. A uniqueness constraint prevents duplicate active reservations for the same student and event.

### Failure handling

Commit the reservation and idempotency result together. On a repeated key, return the stored result. Avoid external network requests while holding the database lock. Return a clear conflict response if the event is full.

### Validation

Run 40 concurrent requests against five remaining seats. Verify that exactly five reservations exist. Retry the same key and confirm it does not create another row. Test a forced rollback and verify that capacity is unchanged. All counts describe a synthetic exercise.

### Trade offs and open questions

Serializing updates to one popular event can increase waiting time. Measure lock duration and latency before considering more complex designs. Payment processing, distributed inventory and production throughput are outside this project.